Сессионное задание

Вопрос 1. Опишите трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны

Опишем классический протокол с доверенным центром распределения ключей (KDC) на примере базовой схемы Kerberos. Он иллюстрирует, как третья сторона помогает двум участникам A и B установить общий сеансовый ключ.

Предпосылки и нотация

- A - инициатор (клиент), B - респондент (сервер/служба), T - доверенная третья сторона (KDC).
- K_AT и K_BT - долговременные симметричные ключи A↔T и B↔T соответственно.
- k - свежий сеансовый ключ A↔B; L - срок его действия.
- N_A - одноразовое число (nonce); T_A - метка времени A.
- E_K(X) - шифрование сообщения X на ключе K.

Роли сторон

- A: запрашивает у T помощь в установлении сеансового ключа с B и доказывает свежесть запроса.
- B: принимает «билет», извлекает k и проверяет, что перед ним именно A и что сообщения не являются повторами.
- T (KDC): генерирует k, упаковывает его для A и B с помощью их долговременных ключей.

Этапы протокола (кто, кому и что отправляет)

1. Запрос A к T

A → T: A, B, N_A

Назначение: A сообщает требуемого корреспондента B и добавляет одноразовый N_A для защиты от подмен/повторов ответа.

2. Ответ T → A

T → A: ticket_B, E_{K_AT}(k, N_A, L, B)

Действия T: сгенерировать свежий k и срок L; подготовить «билет» для B:

- ticket_B = E_{K_BT}(k, A, L)
- для A зашифровать E_{K_AT}(k, N_A, L, B)

Действия A: расшифровать свою часть на K_AT, проверить N_A (свежесть и адресность ответа), запомнить k и L.

3. Передача билета B и аутентификатора

A → B: ticket_B, authenticator

где authenticator = E_k(A, T_A)

Действия B: расшифровать ticket_B ключом K_BT, получить k и L; затем проверить authenticator на k: совпадение имени A и допустимость метки T_A (нет повтора/просрочки).

4. Взаимная аутентификация (опционально)

B → A: E_k(T_A)

Назначение: доказать A владение тем же k и завершить взаимную аутентификацию.

Итог
Участники A и B устанавливают общий сеансовый ключ k, сгенерированный T и безопасно доставленный через долговременные ключи K_AT и K_BT. При выполнении шага 4 достигается взаимная аутентификация.

Свойства и замечания

- Конфиденциальность k: k никогда не передаётся в открытом виде; доступен только A и B (и T как доверенный центр).
- Защита от повторов: свежесть запроса A обеспечивается N_A; свежесть у B - проверкой метки T_A в authenticator.
- Требование синхронизации часов: допустимое окно для T_A должно быть малым; иначе возможны атаки повтором в пределах окна.
- Повторное использование билета: ticket_B годен в течение L; для каждого соединения A формирует новый authenticator, не обращаясь к T, что снижает нагрузку на центр.
- Практическая стойкость: если K_AT/K_BT производятся из паролей, общая стойкость ограничена стойкостью этих паролей.

Источник: Menezes, A., Oorshot, P., Vanstone, S. Handbook of Applied Cryptography [Text] / A.J. Menezes, P.C. Oorshot. 5th edition. - USA.: CRC Press, 2001, §12.3.2, Protocol 12.24 (simplified). Для уточнения отдельных моментов также сверялась с книгой Шнайера «Прикладная криптография». Для ответа использовались учебники из списков литературы, которые представлены в учебных модулях Я.Практикума

Вопрос 2. Приведите формальные модели шифров

Нотация и обозначения (едины для всего раздела)
- M - множество открытых текстов; C - множество шифртекстов; K - множество ключей.
- Gen - генерация ключей; Enc/Dec - шифрование/расшифрование; E_k, D_k - алгоритмы при фиксированном ключе k.
- pk/sk - открытый/закрытый ключ; IV - инициализирующий вектор; r - случайность; $\oplus$ - побитовое XOR.
- Корректность: D_k(Enc_k(m)) = m для всех m $\in$ M и допустимых k $\in$ K.

1. Общая формальная модель криптосистемы
- Формула: S = (M, C, K, E, D); для каждого k $\in$ K заданы E_k: M→C и D_k: C→M, причём D_k(E_k(m)) = m.
- Компоненты - назначение:
- M, C - пространства сообщений и шифртекстов.
- K - пространство ключей (выбирает конкретное преобразование).
- E, D - семейства алгоритмов шифрования/расшифрования по ключу.

- Назначение: базовая абстракция для описания любых шифров и их корректности.

2. Симметричная схема шифрования (SKE)

- Формула: $\Pi = (Gen, Enc, Dec)$; $k \leftarrow Gen()$, $c = Enc_k(m)$, $m = Dec_k(c)$; корректность как выше.
- Компоненты - назначение:
- k - общий секрет отправителя и получателя.
- Enc_k / Dec_k - преобразуют сообщение в шифртекст и обратно.
- m, c - открытый текст и шифртекст.
- Назначение: защита данных при наличии заранее общего секрета; база для блочных и поточных шифров.

3. Асимметричная схема шифрования (PKE)

- Формула: $\Pi = (Gen, Enc, Dec)$; $(pk, sk) \leftarrow Gen()$, $c = Enc_{pk}(m)$, $m = Dec_{sk}(c)$; $Dec_{sk}(Enc_{pk}(m)) = m$.
- Компоненты - назначение:
- pk - открытый ключ для шифрования; sk - секретный ключ для расшифрования.
- Gen - создаёт согласованную пару ключей.
- Enc_pk / Dec_sk - алгоритмы по соответствующим ключам.
- Назначение: защищённая передача без предварительного общего секрета; основа обмена ключами и протоколов.

4. Блочный шифр как семейство перестановок (PRP-модель)

- Формула: для фиксированного размера блока n, $\{E_k\}_{k \in K}$ - семейство перестановок на $\{0,1\}^n$, $D_k = E_k^{-1}$.
- Компоненты - назначение:
- n, $\{0,1\}^n$ - размер и пространство блоков.
- $k \in K$ - выбирает конкретную перестановку E_k.
- D_k - обратное преобразование для расшифрования.
- Назначение: строгая модель блочного шифра; для сообщений произвольной длины используются режимы (CBC, CTR, GCM и др.).

5. Поточный шифр

- Формула: $z = G(k, IV)$, $c = m \oplus z$, $m = c \oplus z$.
- Компоненты - назначение:
- k - секретный ключ; IV/nonce - обеспечивает различие потоков.
- G - генератор ключевого потока (псевдослучайная гамма).
- z - ключевой поток; $\oplus$ - операция шифрования/дешифрования.
- Назначение: шифрование потоков/данных произвольной длины с требованиями к уникальности IV.

6. Одноразовый блокнот (OTP)

- Формула: $M = C = K = \{0,1\}^n$, $c = m \oplus k$, $m = c \oplus k$.
- Компоненты - назначение:

- k - истинно случайный, длиной с сообщение, используется один раз и хранится в секрете.
- m, c - открытый текст и шифртекст; $\oplus$ - операция сложения по модулю 2.
- Назначение: теоретическая модель абсолютно стойкого шифра при строгом соблюдении условий.

7. Детерминированное и вероятностное шифрование

- Детерминированное: c = Enc_k(m) - одинаковые m при одном k дают одинаковые c.
- Вероятностное: c ← Enc_k(m; r) - вводится случайность r/IV, что скрывает повторы сообщений.
- Компоненты - назначение:
- r/IV - свежесть/рандомизация; Enc использует её при шифровании.
- Назначение: вероятностные схемы обеспечивают стойкость уровня IND-CPA и выше; детерминированные применяют ограниченно.

8. Модель шифра с точки зрения стойкости (эксперименты)

- IND-CPA (выбор сообщения): противник выбирает m0, m1, получает шифрование одного из них и пытается угадать бит выбора; преимущество должно быть пренебрежимо мало.
- IND-CCA (адаптивный доступ к оракулу расшифрования): противник имеет доступ к Dec(·) с исключением целевого шифртекста (CCA2 - до и после вызова вызова шифрования-вызова).
- PRG/PRF/PRP: определения стохастических примитивов; потоковые шифры строятся на PRG, блочные - на PRP.
- Назначение: формальные критерии безопасности и сопоставление возможностей нарушителя.

Итоговая сводка: компоненты и назначение

- M, C, K - пространства сообщений/шифртекстов/ключей; каркас любой модели.
- Gen, Enc, Dec - основные алгоритмы; корректность D_k(Enc_k(m)) = m.
- Блочный шифр: {E_k} - семейство перестановок на {0,1}^n.
- Поточный шифр: c = m $\oplus$ G(k, IV); требуются уникальные IV/nonce.
- OTP: идеальная секретность при случайном одноразовом k длины сообщения.
- Детерминированное vs вероятностное: роль случайности r/IV для сокрытия повторов и достижения IND-CPA.
- Модели стойкости: IND-CPA/CCA, PRG/PRF/PRP - стандартные формальные критерии.

Список литературы

- Menezes, A., Oorshot, P., Vanstone, S. Handbook of Applied Cryptography [Text] / A.J. Menezes, P.C. Oorshot. 5th edition. - USA.: CRC Press, 2001.
- Mao, W. Modern Cryptography: Theory and Practice. Prentice Hall, 2004 (рус. пер. «Современная криптография», 2005).
- Smart, N. Cryptography: Theory and Practice. Chapman & Hall/CRC, 4th ed., 2016 (рус. пер. «Криптография», 2005).

Вопрос 3. Приведите принципы построения поточных систем шифрования

Поточные шифры предназначены для шифрования данных в виде последовательности символов, битов или байтов. В отличие от блочных шифров, они не обрабатывают сообщение крупными блоками фиксированной длины, а формируют ключевой поток, который затем накладывается на открытый текст. Общие принципы построения поточных шифров рассматриваются в работах Б. Шнайера, А.А. Молдовяна, М.Е. Масленникова, А.П. Алферова, Н. Смарта и в справочнике Handbook of Applied Cryptography [1-6].

1. Общая модель поточного шифра

Базовая схема потокового шифрования:

$z = G(k, IV)$

$c = m \oplus z$

$m = c \oplus z$

Обозначения: m - открытый текст; c - шифртекст; k - секретный ключ; IV (или nonce) - инициализирующий вектор/уникальный начальный параметр; G - генератор ключевого потока; z - ключевой поток (гамма); $\oplus$ - побитовое XOR.

Смысл: отправитель и получатель, имея один и тот же k и согласованный IV, порождают одинаковый z и, соответственно, корректно шифруют/расшифруют поток данных.

Главная идея поточного шифра состоит в том, что открытый текст складывается с ключевым потоком. Если получатель знает тот же ключ и начальное значение, он может сгенерировать тот же ключевой поток и восстановить сообщение.

Эта модель поточного шифрования описывается в классической литературе по прикладной криптографии и криптографическим алгоритмам [1; 2; 4; 6].

2. Принципы построения поточных систем шифрования

2.1. Принцип формирования ключевого потока

Основой поточного шифра является генератор ключевого потока. Он вырабатывает последовательность битов/байтов, которая используется для шифрования открытого текста.

Формально: $z_1, z_2, \ldots, z_n = G(k, IV)$; каждый элемент шифруется как $c_i = m_i \oplus z_i$.

Для чего нужен принцип (и эффект):

- Сокрытие открытого текста - ключевой поток маскирует исходные данные (конфиденциальность).
- Потоковая обработка - данные шифруются по мере поступления (низкая задержка).
- Высокая скорость - нет ожидания накопления целого блока (производительность).
- Применимость к «живым» потокам - сетевой трафик, аудио/видео, телеком.

Литература: Шнайер; Молдовян; НАС [1; 2; 6].

2.2. Принцип псевдослучайности ключевого потока

Ключевой поток должен быть неотличим от случайной последовательности для полиномиального противника. Если гамма имеет заметные закономерности, нарушитель может использовать их для восстановления открытого текста или ключа.

Свойства хорошего ключевого потока:

- близость к равномерному распределению нулей и единиц;
- отсутствие коротких и регулярных повторов;
- непредсказуемость следующих битов при известном префиксе;
- минимальные статистические зависимости между выходами и внутренними состояниями/регистрами.

Для чего нужен принцип (и эффект):

- Криптографическая стойкость - чем «случайнее» гамма, тем труднее анализировать шифртекст статистически.
- Защита от утечек о структуре сообщения - повторы и шаблоны открытого текста не «просвечивают» в шифртексте.
- Непредсказуемость - по части z нельзя вычислить его продолжение (свойство криптографического PRG).

Литература: Молдовян; Масленников; Smart; НАС [2; 3; 5; 6].

2.3. Принцип зависимости ключевого потока от секретного ключа

Ключевой поток должен определяться секретным ключом: $z = G(k)$ (или в практике $z = G(k, IV)$). Это означает, что без знания k невозможно воспроизвести корректную гамму.

Для чего нужен принцип (и эффект):

- Секретность - только владелец k может породить правильный поток z.
- Управляемость/масштабируемость - один алгоритм работает с множеством ключей для разных пользователей/сеансов.
- Принцип Керкгофса - стойкость не должна зависеть от секретности алгоритма, а от секретности ключа.

Литература: Шнайер; Алферов; Ященко; НАС [1; 4; 6; 7].

2.4. Принцип использования инициализирующего вектора

В практических поточных шифрах используется инициализирующий вектор IV (или nonce).

Формально: $z = G(k, IV)$. Это позволяет при одном и том же ключе k получать разные ключевые потоки для разных сообщений/пакетов.

Для чего нужен принцип (и эффект):

- Предотвращение повтора гаммы - разные IV дают разные потоки z (стойкость к анализу повторов).
- Практическое удобство - не нужно менять секретный ключ для каждого сообщения (разделение сеансов).
- Масштабируемость протоколов - уникальные IV на пакет/кадр/сообщение.

Важное условие (запрет повтора):

Если два сообщения зашифрованы одной и той же гаммой, c1 = m1 ⊕ z, c2 = m2 ⊕ z, то c1 ⊕ c2 = m1 ⊕ m2. Это раскрывает связь между открытыми текстами. Следовательно, для фиксированного k повтор IV недопустим.

Литература: Шнайер; Молдовян; Масленников; HAC [1; 2; 3; 6].

2.5. Принцип большого периода ключевой последовательности

Генератор ключевого потока должен иметь большой период (последовательность не должна быстро повторяться). Период - это длина минимального фрагмента, после которого начинается повтор.

Для чего нужен принцип (и эффект):

- Стойкость к анализу повторов - короткий период облегчает восстановление структуры гаммы.
- Защита длинных сообщений - z не должен повторяться внутри одного сообщения/сессии.
- Устойчивость к статистическим атакам - повторы дают нарушителю дополнительную информацию.
- Пригодность для длительных потоков - важно для сетевого трафика и долгих сеансов.

Замечание: большой период сам по себе не гарантирует безопасность - необходима также криптографическая непредсказуемость (PRG-свойства).

Литература: Молдовян; Масленников; HAC [2; 3; 6].

2.6. Принцип нелинейности преобразований

Многие потоковые шифры используют регистры сдвига, булевы функции, нелинейные фильтры и комбинирующие функции. Требуется достаточная нелинейность: чисто линейные схемы легко анализируются и компрометируются.

Для чего нужен принцип (и эффект):

- Стойкость к линейному анализу - нелинейность разрушает простые линейные соотношения между S_t, k и z.
- Защита внутреннего состояния - нарушителю сложнее восстановить S_t по выходу z.
- Устойчивость к корреляционным атакам - ослабляет связь между выходной последовательностью и отдельными регистрами/входами.
- Повышение криптографической сложности - усложняет модели атак и повышает требуемые ресурсы противника.

Литература: Молдовян; Масленников; Smart; HAC [2; 3; 5; 6].

2.7. Принцип сложности восстановления внутреннего состояния

Поточный шифр имеет внутреннее состояние S_t, которое изменяется на каждом шаге: S_{t+1} = F(S_t, k), z_t = H(S_t). При известном открытом тексте нарушитель может извлечь фрагменты z по формуле z_i = m_i ⊕ c_i. Требование: даже знание фрагмента гаммы не должно позволять восстановить k или всё состояние генератора.

Для чего нужен принцип (и эффект):

- Защита ключа - по выходной последовательности нельзя восстановить секретный k.

- Forward security по выходу - знание части z не позволяет предсказывать будущие биты.
- Backward privacy по состоянию - компрометация текущего S_t не должна автоматически раскрывать всю прошлую гамму.
- Стойкость к атакам по известному открытому тексту - при знании m_i нарушитель не восстанавливает k/S_t.

Литература: Шнайер; Молдовян; НАС [1; 2; 6].

2.8. Принцип синхронизации отправителя и получателя

Для правильного расшифрования отправитель и получатель должны использовать одинаковый ключевой поток. Это требует синхронизации генераторов гаммы.

Различают два основных типа поточных шифров:

Синхронные поточные шифры - ключевой поток формируется независимо от шифртекста.

Самосинхронизирующиеся поточные шифры - ключевой поток зависит от предыдущих символов шифртекста.

Для чего нужен принцип (и эффект):

- Корректность расшифрования - стороны должны породить один и тот же поток z.
- Устойчивость к потерям/ошибкам - самосинхронизирующиеся схемы могут восстанавливаться после потерь.
- Контроль ошибок - поведение ошибок различается и определяется типом шифра.
- Практическая применимость - тип выбирают по свойствам канала/протокола.

Сравнение типов:

- Синхронный: z не зависит от шифртекста; плюс - ошибка в одном бите влияет только на соответствующий бит m; минус - потеря/вставка битов нарушает синхронизацию.
- Самосинхронизирующийся: z зависит от предыдущих символов c; плюс - возможна автоматическая ресинхронизация; минус - ошибка может повлиять на несколько следующих символов.

Литература

Классификация поточных шифров на синхронные и самосинхронизирующиеся приводится в Handbook of Applied Cryptography, а также рассматривается у Б. Шнайера и А.А. Молдовяна [1; 2; 6].

2.9. Принцип эффективности реализации

Поточные шифры применяются там, где важны скорость и низкая задержка. Принцип: архитектура и примитивы должны быть эффективны в ПО и аппаратуре, в т.ч. на ограниченных устройствах и при больших потоках.

Для чего нужен принцип (и эффект):

- Высокая скорость - быстрая обработка данных (через простой раундовый граф/операции add-rotate-xor и т.п.).
- Низкая задержка - шифрование сразу по мере поступления данных (streaming I/O).
- Экономия ресурсов - пригодность для встроенных/мобильных систем.

- Реальное время - устойчивое быстродействие для аудио/видео/сетей.

Литература: Шнайер; Молдовян; Масленников [1; 2; 3].

2.10. Принцип достаточной длины ключа

Пространство ключей должно быть достаточно большим, чтобы полный перебор был практически невозможен. Обозначим множество ключей как K; сложность перебора растёт с |K|.

Для чего нужен принцип (и эффект):

- Стойкость к полному перебору - большое |K| делает перебор нецелесообразным.
- Долговременная защита - создаёт запас стойкости с учётом роста вычислительных мощностей.
- Соответствие практическим требованиям - подбор длины ключа под актуальные стандарты безопасности.

Литература: Шнайер; Алферов; Smart; HAC [1; 4; 5; 6].

2.11. Принцип устойчивости к статистическим и корреляционным атакам

Ключевой поток не должен иметь статистически заметных зависимостей от: секретного ключа, внутреннего состояния, отдельных регистров генератора и предыдущих выходных битов. Иначе нарушитель может восстановить состояние генератора или ключ.

Для чего нужен принцип (и эффект):

- Стойкость к корреляционному анализу - затруднить связывание выходов z с внутренними регистрами/состояниями.
- Стойкость к статистическим атакам - отсутствие полезных закономерностей в z.
- Защита структуры генератора - даже при известном алгоритме не должно быть эксплуатируемых зависимостей.
- Повышение непредсказуемости - выходная последовательность выглядит случайной для противника.

Литература: Молдовян; Масленников; Smart; HAC [2; 3; 5; 6].

2.12. Принцип защиты от повторного использования гаммы

Нельзя использовать один и тот же ключевой поток для разных сообщений. Если гамма повторена: $c1 = m1 \oplus z$, $c2 = m2 \oplus z$, то $c1 \oplus c2 = m1 \oplus m2$ - гамма «сокращается», и открытые тексты оказываются связанными.

Для чего нужен принцип (и эффект):

- Сохранение конфиденциальности - повтор гаммы раскрывает информацию о сообщениях.
- Защита при анализе нескольких шифртекстов - не допускать появления $m1 \oplus m2$ у противника.
- Корректное использование IV/nonce - уникальные значения предотвращают повтор потока.
- Практическая безопасность - даже стойкий алгоритм становится уязвим при повторах гаммы.

Литература: Шнайер; Ященко; Молдовян; НАС [1; 2; 6; 7].

2.13. Принцип корректной инициализации генератора

Перед началом выработки ключевого потока генератор должен быть правильно инициализирован ключом и начальным значением. Часто после загрузки k и IV выполняют «прогрев» состояния.

Формально: начальное состояние $S_0 = Init(k, IV)$, далее генерация $z_t = H(S_t)$.

Для чего нужен принцип (и эффект):

- Скрытие связи между k, IV и первыми битами z - начальное перемешивание уменьшает риск утечек.
- Корректная работа генератора - переход в допустимое внутреннее состояние перед выдачей z.
- Защита от слабых начальных состояний - исключение предсказуемых последовательностей на старте.

Литература: Молдовян; Масленников; Шнайер [1; 2; 3].

2.14. Принцип минимального распространения ошибок

В поточных шифрах ошибка в одном бите шифртекста часто приводит к ошибке только в соответствующем бите открытого текста: $m_i = c_i \oplus z_i$. Если искажён c_i, то искажается только m_i (при сохранённой синхронизации).

Для чего нужен принцип (и эффект):

- Работа в каналах с ошибками - пригодность для ненадёжных каналов связи.
- Локализация ошибок - ошибка не портит весь крупный блок данных.
- Практическая надёжность - важно при передаче потоковых данных.
- Меньшие задержки - нет необходимости повторно обрабатывать большие блоки.

Замечание: преимущественно относится к синхронным потоковым шифрам. В самосинхронизирующихся системах ошибка может затронуть несколько последующих символов.

Литература: НАС; Шнайер [1; 6].

2.15. Принцип разделения функций шифрования и контроля целостности

Поточный шифр обеспечивает конфиденциальность, но сам по себе не защищает от незаметного изменения данных: из-за $\oplus$ изменение бита шифртекста предсказуемо меняет соответствующий бит открытого текста.

Для чего нужен принцип (и эффект):

- Защита от подмены - требуется дополнительный контроль целостности (MAC) или режим AEAD.
- Предотвращение активных атак - противник не должен незаметно модифицировать c.
- Комплексная безопасность - конфиденциальность должна сочетаться с аутентификацией сообщений.

- Практическая надёжность протоколов - на практике применяют AEAD (например, ChaCha20-Poly1305, см. RFC 8439) либо «Encrypt-then-MAC».

Литература: Шнайер; Mao; Smart; HAC [1; 5; 6; 8].

3. Основные подходы к построению генераторов гаммы

3.1. Генераторы на регистрах сдвига

Один из классических подходов - использование регистров сдвига, особенно линейных регистров сдвига с обратной связью. Такие регистры удобны из-за высокой скорости и простоты аппаратной реализации, но сами по себе линейные регистры недостаточно стойки. Поэтому их обычно дополняют нелинейными фильтрами или комбинирующими функциями.

Ключевые моменты:

- Эффективность: регистры сдвига легко реализуются аппаратно.
- Большой период: при правильном выборе параметров можно получить длинные последовательности.
- Основа для генераторов: несколько регистров можно комбинировать для повышения стойкости.
- Необходимость нелинейности: чисто линейные схемы уязвимы к анализу.

Литература
Регистры сдвига и генераторы на их основе рассматриваются у А.А. Молдовяна, М.Е. Масленникова и в Handbook of Applied Cryptography [2; 3; 6].

3.2. Фильтрующие генераторы

В фильтрующем генераторе внутреннее состояние регистра обрабатывается нелинейной функцией: [z_t = f(S_t)], где f - нелинейная фильтрующая функция.

Ключевые моменты:

- Повышение нелинейности: выходная последовательность сложнее связана с состоянием регистра.
- Стойкость к линейному анализу: нелинейная функция затрудняет восстановление регистра.
- Сохранение эффективности: основа генератора может оставаться простой.
- Улучшение статистических свойств: правильно выбранная функция улучшает свойства гаммы.

Литература
Фильтрующие функции и нелинейные генераторы описываются в работах А.А. Молдовяна и в Handbook of Applied Cryptography [2; 6].

3.3. Комбинирующие генераторы

В комбинирующих генераторах используется несколько источников последовательностей, а выход получается с помощью функции объединения: [z_t = f(x_{1,t}, x_{2,t}, ..., x_{r,t})], где x_{i,t} - выходы отдельных регистров или генераторов.

Ключевые моменты:

- Увеличение сложности: несколько генераторов сложнее анализировать, чем один.
- Повышение периода: периоды отдельных последовательностей могут комбинироваться.
- Стойкость: нелинейная функция объединения снижает риск восстановления состояния.
- Гибкость проектирования: можно строить разные схемы под разные требования.

Литература

Комбинирующие генераторы и их криптографические свойства рассматриваются у А.А. Молдовяна, М.Е. Масленникова и в Handbook of Applied Cryptography [2; 3; 6].

4. Сводка принципов: для чего нужен каждый (без таблицы)

- Формирование ключевого потока - маскировать открытый текст; эффект: конфиденциальность и потоковая обработка данных.
- Псевдослучайность гаммы - исключить закономерности в z; эффект: стойкость к статистическому анализу и непредсказуемость.
- Зависимость от секретного ключа - чтобы только владелец k мог получить корректную гамму; эффект: секретность при раскрытом алгоритме.
- Использование уникального IV/nonce - получать разные потоки при одном k; эффект: защита от повторов и атак по c1 ⊕ c2.
- Большой период - избежать повторов в длинных потоках; эффект: пригодность для длительных сессий (при условии криптостойкости PRG).
- Нелинейность преобразований - усложнить математический/корреляционный анализ; эффект: повышение криптографической стойкости.
- Сложность восстановления состояния - по выходу нельзя восстановить S_t/k; эффект: защита ключа и будущих/прошлых сообщений.
- Синхронизация отправителя и получателя - породить один и тот же z; эффект: корректность расшифрования и управляемое поведение ошибок.
- Эффективность реализации - высокая скорость и низкая задержка в ПО/аппаратуре; эффект: применимость в реальном времени и на ограниченных устройствах.
- Достаточная длина ключа - противодействие полному перебору; эффект: вычислительная стойкость с запасом.
- Устойчивость к статистическим/корреляционным атакам - минимизировать утечки о регистрах/состоянии; эффект: криптостойкость потока z.
- Запрет повторного использования гаммы - не допускать одинакового z для двух сообщений; эффект: сохранение конфиденциальности между шифртекстами.
- Корректная инициализация (прогрев) - скрыть связь между k, IV и первыми битами z; эффект: надёжный старт без слабых состояний.
- Минимальное распространение ошибок - локализовать искажения; эффект: устойчивость к ошибкам канала (особенно у синхронных схем).
- Разделение шифрования и контроля целостности - добавлять MAC/AEAD; эффект: защита от активных атак и подмен.

5. Вывод

Поточные системы шифрования строятся на принципе генерации ключевого потока и его наложения на открытый текст. Основная формула: $c = m \oplus z$, где z - гамма, формируемая генератором на основе секретного ключа и, как правило, инициализирующего вектора.

Ключевые принципы построения поточных шифров включают:

- генерацию ключевого потока;
- псевдослучайность гаммы;
- зависимость гаммы от секретного ключа;
- использование уникального инициализирующего вектора;
- большой период последовательности;
- нелинейность внутренних преобразований;
- невозможность восстановления внутреннего состояния;
- синхронизацию отправителя и получателя;
- эффективность реализации;
- достаточную длину ключа;
- устойчивость к статистическим и корреляционным атакам;
- запрет повторного использования гаммы;
- корректную инициализацию;
- контролируемое распространение ошибок;
- дополнение шифрования механизмами контроля целостности.

Эти принципы обеспечивают основные свойства поточных шифров: стойкость, эффективность, быстродействие, удобство обработки потоковых данных и практическую применимость в системах защищённой связи.

Список литературы

Шнайер, Б. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си / Пер. с англ. — М.: ТРИУМФ, 2012. — 610 с.

Молдовян, А.А. Криптография: скоростные шифры / А.А. Молдовян и др. — СПб.: БХВ-Петербург, 2002. — 496 с.

Масленников, М.Е. Практическая криптография. — СПб.: БХВ-Петербург, 2003. — 464 с.

Алферов, А.П. Основы криптографии: учеб. пособие для вузов / А.П. Алферов, А.Ю. Зубов, А.С. Кузьмин, А.В. Черемушкин. — М.: Гелиос АРВ, 2001. — 480 с.

Смарт, Н. Криптография / Пер. с англ. — М.: Техносфера, 2005. — 529 с.

Menezes, A.J., Oorschot, P.C. van, Vanstone, S.A. Handbook of Applied Cryptography. — CRC Press, 2001. — 816 p.

Ященко, В.В. Введение в криптографию: учеб. пособие / Под общ. ред. В.В. Ященко. 4-е изд. — М.: МЦНМО, 2012. — 348 с.

Мао, В. Современная криптография: теория и практика / Пер. с англ. — М.: Издательский дом «Вильямс», 2005. — 768 с.

Вопрос 4. Приведите пример режима шифрования, в котором биективность всех шифрующих отображений базового режима не является необходимой для однозначного расшифрования криптограмм. Ответ обоснуйте.

Краткий ответ

Примером является режим счётчика - CTR (Counter Mode).

В режиме CTR базовое шифрующее преобразование используется только для генерации ключевого потока, а не для непосредственного обратимого преобразования открытого текста. Поэтому для однозначного расшифрования не требуется, чтобы базовое отображение E_k было биективным. Достаточно детерминизма: при одинаковом ключе и одинаковом входе функция выдаёт один и тот же результат.

1. Общая идея режима CTR

Режим CTR превращает блочный шифр или вообще некоторую детерминированную функцию в генератор ключевого потока.

Для каждого блока формируют вход генератора: $X_i = Nonce \ // \ counter_i$ (конкатенация $//$).

Затем вычисляют ключевой блок (гамму): $S_i = E_k(X_i)$.

После этого шифруют: $C_i = P_i \oplus S_i$.

Обозначения:

- P_i - i-й блок открытого текста;
- C_i - i-й блок шифртекста;
- S_i - i-й блок ключевого потока;
- E_k - базовое шифрующее преобразование на ключе k;
- $\oplus$ - побитовое XOR.

2. Механизм расшифрования в CTR

Получатель повторяет генерацию ключевого потока: $S_i = E_k(Nonce \ // \ counter_i)$ и восстанавливает текст: $P_i = C_i \oplus S_i$.

Обоснование: $P_i = (P_i \oplus S_i) \oplus S_i$, а потому что $S_i \oplus S_i = 0$, имеем $P_i = P_i \oplus 0 = P_i$ — восстановление однозначно.

3. Почему биективность E_k не требуется

В режимах ECB/CBC нужно инвертировать шифр: $D_k = E_k^{-1}$ (например, $C_i = E_k(P_i)$, $P_i = D_k(C_i)$), иначе расшифрование невозможно.

В CTR базовое отображение применяется не к открытому тексту, а к счётчику/синхропосылке: $S_i = E_k(Nonce \ // \ counter_i)$, а текст получается XOR-ом: $C_i = P_i \oplus S_i$.

Для расшифрования не нужен E_k^{-1} — достаточно повторно вычислить тот же S_i. Следовательно, для однозначности достаточно знать k, Nonce, порядок $counter_i$ и чтобы E_k была детерминированной. Биективность E_k не обязательна.

4. Обоснование на формальном уровне

Пусть $E_k: \{0,1\}^n \to \{0,1\}^n$ не биективна (есть $X_a \neq X_b$ с $E_k(X_a) = E_k(X_b)$). В CTR открытый текст не извлекается как $D_k(C_i)$, а вычисляется как $P_i = C_i \oplus S_i$, где фиксирован $S_i = E_k(\text{Nonce} \ // \ \text{counter}_i)$.

При фиксированном S_i отображение $P_i \mapsto P_i \oplus S_i$ — биекция, поэтому для каждого C_i ровно один $P_i = C_i \oplus S_i$. Однозначность обеспечивается обратимостью XOR, а не биективностью E_k.

5. Важное уточнение о стойкости

Отсутствие требования биективности относится только к однозначности расшифрования. Для стойкости необходимы:

- непредсказуемость и неповторяемость ключевого потока;
- уникальные Nonce и монотонный counter_i под данным ключом k;
- криптографическая стойкость базового преобразования. При повторе гаммы: $C_1 = P_1 \oplus S$, $C_2 = P_2 \oplus S \Rightarrow C_1 \oplus C_2 = P_1 \oplus P_2$.

6. Итоговое обоснование

В CTR биективность базового отображения не требуется для корректного и однозначного восстановления открытого текста, потому что расшифрование — это XOR с детерминированно воспроизводимым ключевым потоком, а не обращение E_k.

Аналогичный аргумент применим к режиму OFB: ключевой поток порождается как итерация E_k, а расшифрование также выполняется XOR-ом; потому биективность E_k для однозначности не требуется.

Список литературы

- Dworkin, M. Recommendation for Block Cipher Modes of Operation: Methods and Techniques. NIST SP 800-38A, 2001.
- Menezes, A.J., van Oorschot, P.C., Vanstone, S.A. Handbook of Applied Cryptography. CRC Press, 2001.

Вопрос 5. Сколько 64-битовых блоков открытого текста искажается при расшифровании криптограммы симметричного блочного шифра, если в шифрованном тексте исказился один бит и используется режим гаммирования с обратной связью по шифртексту, в котором блоки открытого и шифрованного текста имеют длину 8 бит? Ответ обоснуйте.

Краткий ответ

Если используется режим CFB с размером блока базового блочного шифра 64 бита и размером обрабатываемого сегмента 8 бит, то при искажении одного бита шифртекста при расшифровании искажается $1 + 64/8 = 9$ 8-битовых сегментов открытого текста, то есть 9 байтов. Так как один 64-битовый блок содержит $64/8 = 8$ сегментов, то 9 искажённых байтов затрагивают 2 соседних 64-битовых блока открытого текста. Ответ: искажаются 2 64-битовых блока открытого текста.

1. Логика работы режима CFB

Режим CFB - Cipher Feedback Mode, или режим гаммирования с обратной связью по шифртексту. В этом режиме блочный шифр используется не напрямую для шифрования блока открытого текста, а для выработки гаммы (Z_i), которая затем складывается с открытым текстом по XOR.

Пусть задано: размер блока базового шифра $n = 64$ бита; размер сегмента $s = 8$ бит; R_i - 64-битовый регистр; E_K - шифрование базового шифра; P_i - i-й 8-битовый сегмент открытого текста; C_i - i-й 8-битовый сегмент шифртекста; Z_i - 8-битовая гамма.

1.1. Шифрование в CFB

На каждом шаге: $Y_i = E_K(R_i)$; берутся старшие s бит: $Z_i = MSB_s(Y_i)$ (в нашей задаче $s = 8$); затем $C_i = P_i \oplus Z_i$; регистр обновляется сдвигом на s бит с заведением C_i: $R_{i+1} = shift(R_i, C_i)$.

1.2. Расшифрование в CFB

Используются те же операции шифрования базового шифра: $Y_i = E_K(R_i)$, $Z_i = MSB_s(Y_i)$, далее $P_i = C_i \oplus Z_i$, и регистр обновляется так же: $R_{i+1} = shift(R_i, C_i)$. Ошибочный C_i попадает в регистр и некоторое число шагов влияет на последующую гамму.

2. Как распространяется ошибка в CFB

Пусть в одном 8-битовом сегменте шифртекста C_j исказился один бит, то есть при расшифровании поступает $C'_j = C_j \oplus e$, где e - 8-битовая маска с единицей в одном бите.

2.1. Ошибка в текущем сегменте открытого текста

При расшифровании текущего сегмента используется корректная гамма Z_j (поскольку R_j ещё не содержит ошибочного шифртекста): $P'_j = C'_j \oplus Z_j = (C_j \oplus e) \oplus Z_j = (C_j \oplus Z_j) \oplus e = P_j \oplus e$. Искажается ровно тот бит текущего 8-битового сегмента P_j, который искажён в C_j.

2.2. Ошибка в следующих сегментах

После обработки C'_j ошибочный сегмент попадает в регистр обратной связи. Так как длина регистра $n = 64$ бита, а на каждом шаге он сдвигается на $s = 8$ бит, ошибочный 8-битовый сегмент находится в регистре ровно $n/s = 8$ следующих шагов. Следовательно, ошибка влияет на формирование гаммы для сегментов $P_{j+1}, \ldots, P_{j+8}$. После этого ошибочный сегмент полностью вытесняется из регистра, и начиная с P_{j+9} расшифрование снова корректно.

3. Общее число искажённых 8-битовых сегментов

Искажаются: текущий сегмент P_j и следующие $n/s = 8$ сегментов $P_{j+1}, \ldots, P_{j+8}$. Итого $1 + 8 = 9$ сегментов по 8 бит, то есть $9 \cdot 8 = 72$ бита.

4. Переход к 64-битовым блокам открытого текста

Один 64-битовый блок открытого текста содержит $64/8 = 8$ 8-битовых сегментов. Поскольку искажено 9 сегментов (72 бита), они занимают $\lceil 72/64 \rceil = 2$ блока. Следовательно, затронуты два соседних 64-битовых блока.

Иллюстрация

Пусть 64-битовый блок открытого текста состоит из 8 байтов: B1, B2, B3, B4, B5, B6, B7, B8. Если ошибка пришлась на сегмент, соответствующий байту B3, то искажаются: в текущем блоке - B3, B4, B5, B6, B7, B8 (6 байтов); в следующем блоке — B1, B2, B3 (3 байта). Всего: 6 + 3 = 9 байтов.

5. Итоговое обоснование

В режиме CFB ошибка в одном бите шифртекста влияет: (a) на соответствующий 8-битовый сегмент открытого текста напрямую, и (b) на следующие сегменты через регистр обратной связи. Поскольку размер регистра равен размеру блока (64 бита), а сдвиг - 8 бит, ошибочный сегмент остаётся в регистре 64/8 = 8 шагов. Поэтому всего искажается 1 + 8 = 9 сегментов (72 бита), что затрагивает ceil(72/64) = 2 64-битовых блока.

Ответ

При искажении одного бита шифртекста в режиме CFB с 64-битовым блоком базового шифра и 8-битовыми сегментами при расшифровании искажаются 2 64-битовых блока открытого текста. В более детальной форме: искажается 9 байтов (9 сегментов по 8 бит), что соответствует двум соседним 64-битовым блокам.

Список литературы

- Dworkin, M. Recommendation for Block Cipher Modes of Operation: Methods and Techniques. NIST Special Publication 800-38A, 2001.
- Menezes, A.J., van Oorschot, P.C., Vanstone, S.A. Handbook of Applied Cryptography. CRC Press, 2001.

Вопрос 6. Однонаправленная хэш-функция принимает значения из множества двоичных n-мерных векторов. При каком n случайная пара сообщений составляет коллизию с вероятностью, не большей 10^-9? Приведите ход решения.

Краткий ответ

Минимальное n = 30.

Модель и обозначения

- Предполагаем идеализированную хэш-функцию с равномерным распределением значений на множестве H = {0,1}^n (модель случайной функции/идеального хэша).
- Рассматриваем ровно две независимые случайные строки сообщений M1 и M2; интересует событие коллизии h(M1) = h(M2).

Шаг 1. Вероятностная модель и базовая формула

Для двух независимых сообщений вероятность коллизии равна вероятности совпадения двух независимых равномерных n-битовых значений:

P[collision for 2 messages] = 1 / 2^n.

Шаг 2. Требуемое неравенство

По условию требуется, чтобы эта вероятность не превышала 10^-9:

1 / 2^n ≤ 10^-9.

Шаг 3. Выражаем n через логарифмы по основанию 2

Берём log2 от обеих частей (учитывая монотонность логарифма):

-n ≤ log2(10^-9).

Преобразуем правую часть: log2(10^-9) = -9 · log2(10).

Следовательно,

n ≥ 9 · log2(10).

Численное значение log2(10) возьмём с достаточной точностью: log2(10) ≈ 3.321928094.

Тогда

n ≥ 9 · 3.321928094 ≈ 29.897352846.

Поскольку n - целое число бит, берём наименьшее целое, не меньшее правой части:

n_min = 30.

Шаг 4. Проверка (sanity check)

- Для n = 30: 2^-30 ≈ 9.3132 · 10^-10 ≤ 10^-9 - условие выполняется.
- Для n = 29: 2^-29 ≈ 1.8626 · 10^-9 > 10^-9 - условие нарушается, значит n = 29 недостаточно.

Итак, минимальное n действительно равно 30.

Замечание про «парадокс дней рождений»

- Для пары сообщений используется точная формула 1/2^n, и парадокс дней рождений не требуется.
- Для множества из t сообщений вероятность хотя бы одной коллизии (при малых вероятностях) аппроксимируется «правилом дней рождений»:

P_collision ≈ t(t−1) / 2^{n+1}.

Из неё видно, что для «поиска коллизии» в общем случае нужна примерно 2^{n/2} работа, что объясняет выбор «двойной» длины хэша при целевой стойкости к коллизиям (например, 256 бит → ≈ 128-битная стойкость).

Итог

- Требуемое условие 1/2^n ≤ 10^-9 даёт n ≥ 9·log2(10) ≈ 29.897....
- Минимальное целое n, удовлетворяющее условию, - 30 бит.

Список литературы

- Menezes, van Oorschot, Vanstone. Handbook of Applied Cryptography, §9 (хэш-функции и вероятностные оценки).
- Stinson, D. Cryptography: Theory and Practice (оценки вероятностей коллизий и «birthday bound»).